CRYSTAL
PRACTICE MANAGEMENT

CRYSTAL PM

HOW-TO GUIDE

Connect Crystal Meet rooms to Zoom

Let each room join Zoom meetings through Zoom's Meeting SDK, including meetings hosted by customers and vendors, with no one signing in on the device.

This guide has two halves. The Zoom half is done once for all rooms by the person who administers your Zoom account: two apps in the App Marketplace, one review request, and one Zoom user per room, about 30 minutes at the keyboard. Zoom's review of the app then runs for a few weeks in the background; rooms join meetings on your own account in the meantime. The device half takes about five minutes per room. Rooms still join only when someone presses Join on the room page. After the six steps, **In the meeting** says what the table does during a Zoom meeting, how Zoom's pop-ups are handled, and what to check in the first real meeting.

STEP 1
The SDK app
A Meeting SDK app and its client id and secret.

STEP 2
Zoom's review
Submit the app, unlisted, so rooms may join outside meetings.

STEP 3
The token app
A Server-to-Server OAuth app with two read-only scopes.

STEP 4
A user per room
Rooms join outside meetings as their own Zoom user.

STEP 5
On the device
The credentials file, the installer, the check.

STEP 6
A test meeting
One inside your account, one outside.

BEFORE YOU BEGIN

Access. The Zoom account owner, or an admin whose role has the **Zoom for developers** permission (then **Develop** shows at the top of the App Marketplace) and who may add users. The owner grants that permission at **admin.zoom.us** under **User Management › Roles**: edit the admin's role, then **Role Settings › Advanced features › Zoom for developers**, tick **View** and **Edit**.

Why four parts. Zoom lets an SDK app join meetings hosted by other Zoom accounts only once Zoom has reviewed the app, and, since March 2026, only as a signed-in Zoom user. The SDK app (step 1) joins your own meetings on its own; the review (step 2), the token app (step 3) and a Zoom user per room (step 4) let the room join everyone else's.

Secrets. You will collect a client id and secret (step 1) and an account id, client id and secret (step 3). Keep them out of email and chat; they go straight into a file on each device.

1 Create the Meeting SDK app

- **Sign in to the App Marketplace** at **marketplace.zoom.us** with the account owner's login, or as an admin with the permission above. Open **Develop** at the top right and choose **Build App**. If there is no Develop menu, the permission is missing; see Before you begin.
- **Pick the type.** In the dialog choose **General App** and press **Create**. Zoom folded its old "Meeting SDK app" type into the General App; there is no separate SDK type to look for.
- **Name it and make it account-owned.** On **Basic Information** set the name to **Crystal Meet rooms** and choose **Admin-managed** where the page asks how the app is managed. It belongs to the account; nobody signs in to it.
- **Copy the development credentials.** Still on Basic Information, **App Credentials** shows two pairs, **Development** and **Production**. Copy the Development **Client ID** and **Client Secret**; they go into the credentials file as **sdk_client_id** and **sdk_client_secret**. The development pair joins meetings on your own account straight away. The Production pair is for after step 2.
- **If the page insists on an OAuth Redirect URL** before it lets you continue, enter **https://crystalpm.com/**. The rooms never use it; it only has to be a page on a domain you own.
- **Turn on the Meeting SDK.** Open **Features** and its **Embed** tab (some layouts show Embed under **Surface**), switch **Meeting SDK** on and save. Leave the three switches beneath it off: **programmatic join use case** (the room joins when a person presses Join, as a Zoom user, so no exception is needed), **Request Anonymous Join Exception** (that is for apps that cannot use a ZAK; the rooms use one) and **Use Device OAuth** (the token app in step 3 does that job). The SDK downloads listed there are not needed: the room loads the web SDK itself.
- **Nothing else on the other pages.** **Access** (secret token, event subscriptions, Plugin SDK), **Surface** (home URL, domain allow list, products, in-client features), **Connect** and **Actions and Triggers** all stay as they are; the room never shows anything inside the Zoom client. On **Local Test**, **Add App Now** is harmless either way.

- **Add the scope Zoom insists on.** The build flow will not continue with an empty **Scopes** page. Press **Add Scopes**, search for `zak` and add `user:read:zak:admin` (View a user's ZAK). If it is not offered, search for `token` and add `user:read:token:admin` instead; if the list only has older names, pick **User › View all user information**, the one Zoom's own SDK guide names for admin-managed apps. In **Scope Description** write: "Fetches a conference-room user's ZAK so the room device can join meetings hosted by other Zoom accounts as that user." Add nothing else; every extra scope lengthens step 2.
- **Skip Local Test.** The rooms do not sign in through the app, so it needs no event subscriptions and no "Add App Now" to work.

  You should now see the app under Manage › Built Apps (Zoom also calls this list Created apps) with Meeting SDK on, and have its development client id and secret.

2 Submit the app to Zoom's review, unlisted

Needed only for meetings hosted by other Zoom accounts, which is most customer and vendor meetings. Zoom's rule: an SDK app may join meetings outside the account that created it only after Zoom has reviewed the app. Reviewed does not mean public; the app can be unlisted afterwards. Zoom asks developers to submit at least a month ahead, so do this the same day as step 1.

- **Ask to publish first.** A new app is private and shows "Ready for local test" with no way to submit. In the Marketplace open **Developer › Created apps**, press the **⋯** menu at the end of the app's row and choose **Request to publish** (older layouts: **Manage**, the app, **Activation**, **Share your app with others › Change Now**, then **OK** in the **Enable Publishing** window). The left menu gains **App Listing**, **Technical Design Document**, **Beta Test** and **Publish**, and the **Production** switch at the top shows the fields they need.
- **Fill in the listing** under **Production › App Listing**. The Publish page lists what is still missing, in five groups: Basic Information, App Information, Links & Support, EU & Discoverability (where **Set my app as 'Unlisted'** lives) and Technical Design. Ready-made text for every field, plus an icon and a gallery image, are in `deploy/zoom-marketplace/` in the repository. In short: a 160 by 160 icon, a short and a long description, for example "Crystal PM's conference-room devices join Zoom meetings from the room's TV. Used only inside Crystal PM.", one image for the gallery, the company name, a developer contact mailbox someone reads, and a category. Where it asks for a support address and privacy policy, use pages on `crystalpm.com`.
- **Write the Technical Design Document.** Zoom will not accept a submission without it. A page is enough: what the app is (a Raspberry Pi behind a conference-room TV that joins meetings as a participant when a person presses Join), what data it touches (a ZAK for the room's own Zoom user, nothing stored beyond the device), and that there are no outside users.
- **The EU fields.** **App Listing › EU & Discoverability** asks for business details down to a bank account's last digits and an identification document. Those are the EU Digital Services Act's "trader" details for apps offered to EU users; an internal device app is not a trader. Email `integration.testers@zoom.us` saying so and asking for the fields to be removed from the build flow, which Zoom's documentation offers, or turn off **List my app in the EU** in that section. The exact wording is in `deploy/zoom-marketplace/listing.md`.

- **Verify the domain when asked.** Every web address in the listing must be on a domain Zoom has verified for your account; the **Publish** page offers a DNS record, a file on the site or a meta tag. Whoever manages crystalpm.com can do it in minutes.
- **On the Publish page** pick the audience **External Zoom users**: that is the choice that goes through Zoom's review, which is what lets the rooms join other accounts' meetings, including ones a customer invites the room to. It does not hand the app to anyone: it stays unlisted after the approval, and it has no screens or sign-in of its own, so it is useless without the credentials on your devices. **Users within my Zoom account** skips the review and would leave the rooms limited to meetings hosted by your own account. In the release notes for the reviewer say the app runs on a Raspberry Pi behind a conference-room TV, joins as an ordinary participant when a person presses Join, has no sign-in flow and no outside users, and is device-specific; offer a short screen recording of a join, or a live session with the review team. Write "none" where it asks for test logins, choose whether Zoom activates the app on approval, accept the Marketplace Development Agreement and submit.
- **After the approval,** on **Manage › Created Apps** use the app's menu to **Set my app as 'Unlisted'** so it never appears in the public Marketplace, then copy the **Production** Client ID and Client Secret from App Credentials into each room's credentials file in place of the development pair (step 5) and restart the service. Zoom's reviewers approve the production pair; the development pair stays limited to your own account's meetings. Zoom's reviewers reply by email with questions; answer them quickly to keep the clock moving.

  You should now see the app's status as In review, and later Approved. Until then rooms join meetings hosted on your own account only.

3 Create the Server-to-Server app for tokens

- **Build it.** **Develop › Build App**, choose **Server-to-Server OAuth** and press **Create**; name it `Crystal Meet room tokens`.
- **Copy the credentials** from **App Credentials**: **Account ID**, **Client ID** and **Client Secret**.
- **Fill in Information.** Zoom will not activate the app without a short description, the company name and a developer contact name and email. Leave event subscriptions off on the **Feature** page.
- **Add two read-only scopes.** Under **Scopes › Add Scopes**, search for `zak` and add `user:read:zak:admin` (View a user's ZAK), then search for `token` and add `user:read:token:admin` (View a user's token). Zoom's docs name the first for ZAKs and the second for the token request the room makes, so add both; nothing else is needed. The list only offers scopes the signed-in person's own role allows, so if either is missing, do this as the account owner.
- **Activate the app** on the **Activation** page. Tokens only work while it is active.

  You should now see the app listed as active, with the account id, client id and secret saved with the step 1 values.

4 Create a Zoom user for each room

- **Add the user.** In `admin.zoom.us` go to **User Management › Users › Add Users**. Use a mailbox you can read, for example `room1@crystalpm.com`, and the **Basic** user type, which is free on a paid account.
- **Activate it.** Open the invitation email in that mailbox and finish the sign-up.
- **Name it after the room.** In the user's profile set the display name to the room's name, for example `Room 1`. That is the name hosts see when the room joins.
- **Repeat** for Room 2 and Room 3.

You should now see one active Basic user per room, each named after its room.

> **GOOD TO KNOW**
>
> The room user never hosts anything and needs no licence. If you skip steps 2 to 4, rooms can still join meetings hosted on your own Zoom account; they just cannot join meetings hosted by other companies.

5 Put the credentials on each device

- **Make the file.** Copy `deploy/rooms/zoom-credentials.example.json` to `zoom-credentials.json` and fill in the six values: the step 1 client id and secret (the development pair until step 2 is approved, the production pair after), the step 3 account id, client id and secret, and that room's user as `room_user`. Each room gets its own file because `room_user` differs.
- **Copy it to the device**, for example `scp zoom-credentials.json pi@crystal-meet-room-1.local:~/`.
- **A device you are installing now.** Add `--zoom-credentials ~/zoom-credentials.json` to the installer command from the setup guide.
- **A device that is already installed.** Type `sudo install -o pi -g pi -m 600 ~/zoom-credentials.json /etc/croom/zoom-credentials.json` (your username instead of pi), then `sudo systemctl restart croom`.
- **Check it.** Type `/opt/croom/venv/bin/croom --check-zoom -c /etc/croom/config.yaml`. It reads:

```
Zoom Meeting SDK: app AbC123xYz, signature minted (Zoom checks it only when a meeting is joined)
Zoom account: server-to-server token obtained for account 9XyZ12ab
Zoom room user: room1@crystalpm.com, ZAK obtained (valid 2 hours)
Ready: this room can join meetings on your own Zoom account now; meetings hosted by other
accounts also need Zoom's approval of the SDK app, and the first outside-hosted meeting proves it.
```

- **Delete the copy in your home folder.** Type `rm ~/zoom-credentials.json`.

You should now see the check end with "Ready".

6 Join a test meeting

- **Inside your account.** Have a colleague start a Zoom meeting and paste its invitation link, passcode included, into the room page's **Join with a link**, or book it on the room's calendar and press **Join now**. To join with the meeting id instead, type it in the same box: a **Passcode** field appears under it, so type the meeting's passcode there and press **Join**. A booking whose Zoom link has no passcode in it is joined with the one written in its invitation text ("Passcode: 123456"). Zoom's meeting appears on the TV within about fifteen seconds. This works from day one, with the development pair.
- **Outside your account,** once step 2 is approved and the production pair is on the device. Join a meeting hosted by someone outside the company, or by a personal Zoom account. The room appears in the participant list under its own name, for example Room 1. If the host uses a waiting room, the room page says it is waiting until the host admits it. This step is the proof for the outside-host route, which has not yet been verified against a live meeting: if Zoom refuses with an authorization message even though the app is approved and the check passed, tell whoever set this up; the fallback is fetching the ZAK through the SDK app's own OAuth.
- **Controls.** From the table page press **Mute**, **Turn camera off**, then **Leave** and **Tap again to leave**.

You should now see both meetings join, the room named after itself in the outside one, and the TV return to the room's idle screen after leaving.

AFTER THE TEST MEETING

In the meeting

While the room is in a Zoom meeting, the table's meeting screen shows the meeting as Zoom has it and offers only what Zoom's client offers the room right now: a control whose button Zoom does not draw, or that the room's role does not allow, is not drawn at all. A change someone else makes shows within a few seconds: the room page asks the device every two seconds, and the device reads Zoom whenever something changes and every five seconds. This section says what the table does, how Zoom's pop-ups are handled, and what to check and send back from the first real meeting.

The controls on the table

- **First row.** **Mute** or **Unmute**, **Turn camera off** or **Turn camera on**, **Raise hand** or **Lower hand**, **Reactions** and **Leave**. Each button says what a tap does and shows what Zoom has: when the host mutes the room, **Mute** turns into **Unmute**. **Reactions** replaces the second row for ten seconds with **Thumbs up**, **Clap**, **Heart**, **Laugh**, **Wow** and **Celebrate**, and **Cancel**. They go to Zoom by Zoom's own names, `Thumbs Up`, `Clap`, `Heart`, `Joy`, `Open Mouth` and `Tada`, the list under **controls › reactions** in the room's config.
- **Second row.** **Captions on** or **Captions off**, **Gallery view** or **Speaker view**, **Hide self view** or **Show self view**, **Participants (7)**, **Chat (2)** with the number of messages not yet read, **Host**, drawn only while Zoom says the room is host or co-host, and **TV**, the room's own button for its TV's remote, which is always the last. The view button names the view a tap switches to.

- **Participants.** Everyone in the meeting, with marks for host, raised hand, muted and video off; the room itself says "(this room)". For a host or co-host each other person has **Mute** (not shown for someone already muted) and **Remove**, and people in the waiting room are listed apart under **Waiting room**, each with **Admit**. The count on the button leaves the waiting room out, and the list shows at most 50 people (the count is exact).
- **Chat.** The last 20 messages other people sent, newest at the bottom, each with its sender and time. It is read-only: there is no way to reply from the table in this version. Opening it marks what is there as read.
- **Host.** **Mute all**, **Ask all to unmute**, **Start recording** or **Stop recording**, **Lock meeting** or **Unlock meeting**, and, for the host only (a co-host has no End), **End meeting for everyone**. Zoom's own refusal (the account does not allow it, say) comes back in Zoom's words.
- **Back, and the two idle minutes.** Participants, Chat and Host each have **Back**, and each returns to the meeting screen by itself after two idle minutes (a tap restarts the wait). **Camera** and **Upcoming meetings** stay where you leave them. A dialog that needs a tap brings the table back to the meeting screen at once.
- **Second taps.** **Leave** becomes **Tap again to leave**, **Remove** becomes **Tap again to remove** and **End meeting for everyone** becomes **Tap again to end**: the second tap must come within five seconds, and a tap in the half second after the first does not count, so a double tap on the glass confirms nothing.
- **What Zoom limits.** Zoom takes one mute a second, and one mute all, ask to unmute, remove, record or lock about every five seconds; a tap inside that window answers `Zoom ignored muteAll (it takes one a few seconds after the last); try again in a moment`, naming Zoom's call. A host's calls are made one at a time, so a second tap waits for the first. A control the page cannot carry out answers in words under the title and the meeting goes on, for example `Zoom's captions button is not on the screen yet; try again in a moment` or `Zoom did not switch captions`.

Zoom's dialogs and notices

- **What the room does.** Zoom draws pop-ups over the meeting (a recording or AI Companion notice, the host asking the room to unmute, a breakout room invitation) and the TV has no mouse, so the room reads each one, its title, its words and its buttons, and a policy decides. Nothing is ever pressed except by that policy or by a tap on the table, and the device presses at most one notice's button a second.
- **Pressed at once.** A dialog with exactly one button whose title or words contain one of the phrases under **dialogs › acknowledge** in the room's config (in any case) is pressed at once, and the sentence of it that says what the meeting is doing (recorded, transcribed, AI Companion) stays under the meeting's title for the rest of the call. The defaults, which the installer and the three room files write out, are `is being recorded`, `is being transcribed`, `AI Companion`, `Zoom AI is on`, `has been muted`, `has been unmuted`, `recording has started` and `recording has stopped`. Edit the list in `/etc/croom/config.yaml` and restart with `sudo systemctl restart croom`. A room installed before this release has no such lines and runs the defaults; add them under **meeting** only to change something.

- **Waiting for a tap.** Every other dialog (more than one button, or words that are not on the list, such as the host asking the room to unmute) shows on the table as a card above the controls, with its title (**Zoom asks** when it has none), its words and one button for each of Zoom's, in Zoom's order, the last one highlighted. Zoom puts the cancelling button first, so recording consent reads **Leave Meeting**, **Got it**. Nothing is pressed until someone taps one, which presses it on the TV. The card's buttons ignore a tap for half a second after it appears or changes, so a tap meant for something under it does not answer the dialog.
- **Everything waits.** `ask_everything: true` under **dialogs** (default `false`) makes every dialog wait for a tap, the notices on the list too.
- **The notice line.** Under the meeting's title each disclosure is its own sentence, in the order they came, and one that is already there is not added again; a notice that only says a state changed (`has been muted`, `has been unmuted`, `recording has stopped`, `transcription has stopped`) adds nothing to the line: the device reads Zoom again (the Mute button shows a mute) and what the meeting disclosed stays. `transcription has stopped` is not on the default list, so it waits on the table until someone taps it; add it to the list to have it pressed at once.
- **When a press fails.** A notice whose button the room cannot press is not left on the TV with nobody to answer it: it waits on the table like any other dialog, its words are on the notice line already, and the journal says so. (A press fails when Zoom does not take it or the button is disabled.)
- **Mistakes in the config.** A mistake under `dialogs:` or `controls:` costs only its own setting, with a warning in the journal: a value that is not a list (`acknowledge: no`, which YAML reads as false) is replaced by its default; an `ask_everything` that is not true or false is read as `false` (the words yes, no, on and off, in quotes, are read as they say); an unknown key is ignored; a section that is not a set of settings gives the defaults. Without a warning: a bare word is a one-item list, a setting with nothing after it takes its default, and `acknowledge: []` acknowledges nothing, so every dialog waits. Anywhere else in the file an unknown key makes the agent set the whole file aside (it prints `Warning: Failed to load config from …`) and start on its built-in defaults.
- **What the journal says.** A stock install's agent writes warnings and errors to the journal and nothing else, with one exception: every dialog. `sudo journalctl -u croom --since "1 hour ago"` shows a line for every dialog, whether or not the agent runs with -v: `Zoom dialog: 'Recording consent' 'This meeting is being recorded.' buttons ['Leave Meeting', 'Got it']`, and then `Acknowledged Zoom's dialog with 'OK'` or `The dialog waits on the table`. The dialogs of the call that is going on are also in `dialogs_seen`, with or without the journal. The warnings mean:
- `Could not press 'OK' on Zoom's dialog '…' '…' (the reason): it waits on the table` The room's press did not take; the dialog is on the table for a tap.
- `meeting.dialogs.acknowledge should be a list, not …: using the defaults` The same for `meeting.controls.reactions`, for `ask_everything` (true or false), for `meeting.dialogs: ignoring …` (a key the section does not know) and for a section that is not a set of settings: the line of the config it names is wrong, and the default is in force (an unknown key is simply ignored).
- `Could not read the meeting state: …` The device could not read Zoom's page (it did not answer within 10 seconds, say); the table keeps what it last had.

- `Could not carry out raise hand: …` A tap failed for a reason that is not Zoom's own refusal (the page itself failed); the table shows the same words.
- `Could not take an event from the page: …` The page sent the device something it could not read (a dialog or a chat message in a shape it did not expect): send the line back.
- `Could not read the meeting page: …` The diagnostics route could not read the page (the meeting ended as it was asked, or the page did not answer); it answers 409 with the same words.
- `A step the provider took on its own failed: …` One of the device's own background steps (pressing a notice, reading the meeting again) failed: send the line back.
- **A refusal Zoom gave a tap.** `The raise hand control was not carried out: …` Zoom's own refusal of a tap, in Zoom's words. This is an INFO line, and the journal holds INFO lines only while the agent runs with -v; the same words are on the table. To see them for a while, add `-v` after `croom.core.agent` on the `ExecStart` line of `/etc/systemd/system/croom.service`, so that it reads `ExecStart=/opt/croom/venv/bin/python -m croom.core.agent -v -c /etc/croom/config.yaml`; then run `sudo systemctl daemon-reload` and `sudo systemctl restart croom`, and take the -v out the same way afterwards, because an agent run with -v writes a line for everything it does.

The diagnostics route

`http://<device>:8080/api/meeting/diagnostics` answers only while the room is in a meeting (otherwise 409, Not in a meeting); each call briefly opens and closes the Reactions picker and the View menu on the TV, so it is a support route, never polled (it lists what they hold). It answers in JSON:

- `user_keys` and `attendee_keys`: the field names Zoom gives the room's own record and the attendees' (every record's, each once).
- `state` and `meeting`: what the device read: the room's mute, video, hand and role, the roster with its waiting-room flags, the buttons the device found, recording and lock.
- `buttons`, `video` and `popovers`: the footer's buttons and the View widget's, the video button as the device reads it, and the items of the Reactions picker and the View menu.
- `dialogs` and `dialogs_seen`: the dialogs on the screen now, and every dialog of this meeting (the last 50: title, text, buttons, label, at, pressed, closed). `pressed` is the button the device pressed on it, whoever asked, and `closed` says Zoom has taken it away: pressed and closed is answered, closed alone went by itself, neither is waiting.
- `sdk_version` and `page`: the version of Zoom's Meeting SDK the page loaded, and the page's title.

The inventory run for the first real meeting

This checklist is for the first real meeting a room joins on a device, and it comes out of this guide once the labels and wordings it collects are pinned in the code. The button labels the device looks for and the words of Zoom's notices were read from Zoom's 6.5.0 web client, not from a live meeting, so that meeting is also an inventory. You need a second person on another Zoom account, who will start a recording, ask the room to unmute, raise a hand, wait in the waiting room, send a chat message, turn on AI Companion and open breakout rooms; for the checks that need it the room is host or co-host. Open `http://<device>:8080/api/meeting/diagnostics` in a browser on the office network at each point below, and read the journal afterwards. The checks are numbered in the order a meeting meets them.

1. **The join.** Join with the room set to join muted (`mic_default_on: false`) and watch the TV. The View menu flashes on the TV for a moment just after the room joins: the device opens it once, reads which view is showing and whether self view is hidden, and closes it with nothing picked, so the table starts from what the client has; it should not get in the way of the meeting. The table should show Unmute within a few seconds, and `state` should say `muted: true` : Zoom's own record of the room can lag the mute the device sends at the join, and this is where to see how long. The journal line `The microphone was not muted at join` means the mute itself was refused.
2. **The footer.** `buttons` lists the label and text of every button in Zoom's footer, and of the View widget's (marked `where: view widget`). Check that Chat and CC (closed captions) are among them, and that the words match what the device looks for: `stop my video` or `start my video` for the camera, `reaction` in the Reactions button's label, `view` in the View button's, and, for captions, a label with caption and show or hide (or closed caption). A control whose button the device cannot find is not drawn on the table, so a control missing from the table means a label here that the code does not know.
3. **The Reactions picker.** `popovers` lists what each menu held when the device opened it: text, label, tooltip, class and whether it is disabled. In `reactions` compare the six names above and a `Raise Hand` row. Then double-tap a reaction on the touchscreen: the row closes at the pick, and the second tap must land on nothing.
4. **The View menu.** In `view` the items should be `Speaker View` , `Gallery View` and `Hide Self View` (`Show Self View` when hidden), the checked view having the class `full-screen-widget__pop-menu--checked` .
5. **A recording banner.** Have the second person start a recording: its real title and words appear in `dialogs_seen` . Compare them with the acknowledge list (does it hold `is being recorded` or `recording has started` ?) and see on the table that its sentence stays under the meeting's title. Stop the recording and read the stop notice's real wording too: it should be one of the stop phrases the device knows, so that it adds nothing to the line.

6. **The consent dialog.** When a recording starts Zoom asks participants to consent: the room should show a card with two buttons, Leave Meeting then Got it, with Got it highlighted, and nothing should be pressed on the TV until someone taps Got it (`dialogs_seen` shows the same buttons in Zoom's order). Double-tap Got it: the card answers once, and the second tap must not land on a control under it.

7. **The host's request to unmute.** Have the host ask the room to unmute. Zoom draws a dialog with a choice, whose words are not on the acknowledge list, so it should wait on the table as a card with Zoom's own buttons; its real title, words and buttons are in `dialogs_seen` . Tap the button that unmutes, and `state` should say `muted: false` .

8. **A raised hand.** Have the second person raise a hand: in `meeting › participants` the person shows `hand: true` and the Participants screen marks them Hand raised. The device reads an attendee's hand from `raiseHand` , `bRaiseHand` or `bHandRaised` , and which of them Zoom really uses is not confirmed: `attendee_keys` shows it. Then tap Raise hand on the table: `state` should say `raisedHand: true` and the button should say Lower hand.

9. **A person in the waiting room.** With the room as host or co-host and the waiting room on, have the second person join: in `meeting › participants` the person shows `waiting: true` (the code reads `isHold` , `waiting` , `bHold` or `isInWaitingRoom`). `attendee_keys` names the fields Zoom really set, and the table lists the person under Waiting room with Admit and leaves them out of the Participants count. If they are not in the attendee list at all, say so.

10. **The AI Companion notice.** Have the host turn on AI Companion: the notice's real title and words appear in `dialogs_seen` . The default list holds `AI Companion` and `Zoom AI is on` : with one button it should be pressed at once and its sentence should stay under the meeting's title; with two buttons, or words the list does not hold, it waits on the table as a card. Either way, send its words back.

11. **A breakout invitation.** Have the host open breakout rooms and invite the room: the invitation has a choice in it, so it should wait on the table as a card with Zoom's own buttons, and its real words and buttons are in `dialogs_seen` . Tap the button that declines it.

12. **A modal with no words.** Some Zoom modals have a title and buttons but no body. Look through `dialogs_seen` for an entry with empty `text` ; its `label` is the aria-label of the modal's root, which the device uses for the words only when it begins with the title.

13. **Chat and its clock.** Have the second person send a chat message, then compare `meeting › chat › messages` in `/api/status` , whose `at` is UTC, with the Pi's clock (`date -u`); the Chat screen shows the same moment in the table's local time. A message Zoom gives no usable time for is stamped with the moment it arrived.

14. **Recording from the Host screen.** With the room as host, tap **Start recording**, then **Stop recording**. After each tap the device waits three seconds (`RECORD_MS`) for Zoom to confirm; if cloud recording takes longer the table says `Zoom did not start recording` although it started, so send that back and the wait can be raised.

15. **A rename.** The table has no rename button, but the route has one. Zoom answers a participant's own rename with an error (neither host nor co-host) and then with success, and the device waits for the success: type `curl -s -X POST -H 'Content-Type: application/json' -d '{"name":"rename","value":"Room 1 TV"}' http://<device>:8080/api/meeting/control`, then check that the name changes in Zoom's own list and that the answer is a 200, not a 409. Zoom takes one rename about every five seconds, like the host's calls.

16. **The host's calls and the SDK's own words.** From the Host screen try **Mute all**, **Ask all to unmute** and **Lock meeting**. When Zoom does not answer a host call, its SDK gives up after ten seconds with `OPERATION_TIMEOUT`, which the device words as `Zoom did not confirm muteAll in time`; a call Zoom never answers at all ends after 12 seconds as `Zoom did not answer muteAll in time`. Send back any you see, with the call that was made.

17. **The room's own record.** `state` and `meeting › me`: `userId` should be a number, `muted` and `videoOn` true or false, and `isHost` and `isCoHost` right for the room's role. Zoom's record of a participant who is an assistant shows no userId and the name "You are a assistant": if the room's record looks like that, send it back, because without an id the room cannot be told from the others in the list.

18. **Leave.** Tap **Leave**, then **Tap again to leave**, and the TV returns to the idle screen. Double-tap Leave quickly first: the second tap counts only half a second after the first, within five seconds, so a double tap confirms nothing; the same holds for Remove and End meeting for everyone on the Host screen. If a double tap does something it should not, or a real second tap is lost, say which button. Zoom's own Leave Meeting button, on the consent card, leaves the call with no second tap, so it comes last of all.

What to send back. Send back the diagnostics JSON from each point and the journal lines: save the first with `curl -s http://<device>:8080/api/meeting/diagnostics > diagnostics-1.json` and the second with `sudo journalctl -u croom --since "2 hours ago" --no-pager | grep -E "Zoom dialog|Acknowledged|waits on the table|WARNING|ERROR"`, and say what the table showed at each point. Nothing is pinned from memory: the acknowledge list and the stop phrases are refined from what comes back, and the button labels in the code follow the labels in your diagnostics.

IF SOMETHING IS OFF

Troubleshooting

There is no Develop menu in the Marketplace

The signed-in admin's role lacks the **Zoom for developers** permission. The account owner adds it under **User Management › Roles** (Role Settings › Advanced features), or does steps 1 to 3 personally.

The check says "not configured"

It names the value that is missing or still a placeholder in `/etc/croom/zoom-credentials.json`. The four server-to-server values and **room_user** go together: fill in all of them or none.

The check says Zoom refused the credentials

The account id, client id or secret of the step 3 app is wrong, or the app is not activated. Copy them again from App Credentials and check the Activation page.

The check says no such user, or a scope is missing

The **room_user** must be an active user on this Zoom account. "Lacks a scope" quotes the scope Zoom wants; add that scope to the step 3 app under **Scopes** and activate it again.

The room page says the meeting is not authorized

The meeting is hosted by another account. Three things must all be true: the SDK app is approved (step 2), the device holds its production pair, and the file has the server-to-server values and **room_user**. Fix what is missing, restart the service and run the check. If all three are true and Zoom still refuses, the outside-host route needs the fallback described in step 6.

Wrong passcode or meeting not found

Paste the full invitation link with its passcode part, or type the passcode in the **Passcode** field that appears under the link box for a meeting id or a link without one. After a "Couldn't join" that says "Zoom needs the meeting passcode", the card keeps the link and shows the field: type the passcode there and tap **Join again**. Zoom's exact reason shows on the room page.

The room joins muted, or a tap on the table says "try again"

The table shows mute and camera as Zoom has them, and a change the host makes shows within a few seconds. At join the room's camera setting (**camera_default_on** in the room's config) applies both ways: Zoom's video is started or stopped to match it. The microphone setting (**mic_default_on**) applies only towards muted: a room set to join muted is muted, but when the room is set to join with the microphone on and Zoom has it muted (the host's mute on entry, for example), the room stays muted and the table shows **Unmute**, one tap away. A tap that Zoom cannot carry out (the camera still starting, no camera Zoom can use, a host who does not allow participants to unmute) answers in words on the meeting screen, Zoom's own where Zoom gave them, and the meeting goes on. To see what the device's page sees (Zoom's field names and its buttons' wording) during a meeting, open **http://<device>:8080/api/meeting/diagnostics** from any browser on the office network and send the result with your report.

The TV stays dark and the page says the SDK did not load

The device could not reach **source.zoom.us**. Check the device's internet access and any web filter, then try again.

GOOD TO KNOW

The room joins meetings as its own Zoom user through Zoom's supported Meeting SDK, so it is never mistaken for a bot. To take that away, deactivate the two apps or the room users in Zoom.